Multi-Admin Group Chat Architecture Plan

This document outlines the system architecture, database changes, and logical workflows required to implement a robust, decentralized Multi-Admin Group Chat model in Qbase.

---

1. System Requirements & Schema Evolution

To allow multiple group administrators, the schema must evolve from a single string `adminId` to a dynamic array of strings `adminIds`.

1.1 Appwrite Cloud Collection Changes (`chats` Collection)

• **Deprecate**: `adminId` (String, length 36).
• **Add Attribute**: `adminIds` as a **String Array** (size 36 per element, array=True, required=True).
• **Security Rule**: Document Level Security (DLS) remains active, allowing participants to read/update the metadata.

1.2 Room SQLite database Changes (`ChatEntity` Table)

• **Table Definition (`chats`)**:

```
@Entity(tableName = "chats")
data class ChatEntity(
    @PrimaryKey val chatId: String,
    val chatName: String?,
    val isGroup: Boolean,
    val participantIds: String, // Comma-separated
    val adminIds: String,       // Comma-separated list of admin UIDs
    val isBlocked: Boolean = false,
    val isReported: Boolean = false,
    val isMuted: Boolean = false,
    val unreadCount: Int = 0,
    val lastUsedKeyFingerprint: String? = null
)
```

• **Database Version**: Increment to `2` inside `ChatDatabase.kt` to trigger Room's auto-recreation of tables.

---

2. Dynamic Privilege & Administrative Logic

All administrative calls within the view models and synchronization engines must adapt to use comma-separated admin checking logic.

2.1 Extension Privilege Evaluator

```
fun ChatEntity.isAdmin(userId: String): Boolean {
    return if (!isGroup) false
    else adminIds.split(",").map { it.trim() }.contains(userId)
}
```

2.2 Promoting a Participant

Any active administrator can promote another group participant to admin status:

```
suspend fun promoteToAdmin(chatId: String, targetUserId: String) {
    val chat = chatDao.getChatById(chatId) ?: return
    if (!chat.isAdmin(currentUserId)) return

    val currentAdmins = chat.adminIds.split(",").map { it.trim() }.toMutableList()
    if (!currentAdmins.contains(targetUserId)) {
        currentAdmins.add(targetUserId)
        val updatedAdminIdsStr = currentAdmins.joinToString(",")

        // Push update to Appwrite Cloud & save to local Room
        chatRemoteRepository.updateAdminListOnRemote(chatId, currentAdmins)
        chatDao.insertChat(chat.copy(adminIds = updatedAdminIdsStr))
    }
}
```

2.3 Resiliency Rules (The "Last Admin" Protocol)

If an administrator leaves the group:

• **Rule 1**: If there are other administrators left, simply remove the leaving admin.
• **Rule 2**: If the leaving admin was the *last* admin, automatically promote the oldest remaining participant to administrator.
• **Rule 3**: If the group becomes completely empty, delete the group metadata completely from the Appwrite cloud.